

Анализ строковых сортировок A1

1 Введение и теоретическая справка

В данном отчете проводится сравнительный анализ классических и специализированных алгоритмов сортировки строк. Стандартные алгоритмы (QuickSort, MergeSort) работают за $O(N \log N)$ операций сравнения. Однако для строк сравнение двух элементов само по себе занимает $O(L)$ времени в худшем случае, где L — длина строки. Это приводит к общей асимптотике $O(N \cdot L \log N)$.

Специализированные алгоритмы учитывают внутреннюю структуру строк (префиксы, посимвольное сравнение), что позволяет существенно сократить количество лишних обращений к одним и тем же символам.

1.1 Теоретические оценки

- **Standard QuickSort & MergeSort:** Асимптотика посимвольных сравнений $O(N \cdot L \log N)$. Алгоритмы многократно сравнивают общие префиксы строк при рекурсивных вызовах.
- **Ternary String QuickSort (A1q):** На каждом шаге выбирается опорный символ, и массив делится на 3 части: $<$, $=$ и $>$. Для средней части мы переходим к следующему символу. Это гарантирует, что каждый символ строки сравнивается конечное число раз. Сложность: $O(N \log N + N \cdot L)$.
- **String MergeSort с LCP (A1m):** Модификация сортировки слиянием. Для каждой пары строк алгоритм запоминает длину наибольшего общего префикса (LCP). При слиянии мы пропускаем уже известные совпадающие префиксы. Сложность: $O(N \log N + N \cdot L)$.
- **MSD Radix Sort (A1r):** Поразрядная сортировка со старшего разряда. Использует сортировку подсчетом для текущего символа. Алфавит состоит из 74 символов. Общая сложность: $O(N \cdot L)$. Проблема алгоритма — большие накладные расходы на создание массивов (корзин) для маленьких подмассивов.
- **MSD Radix + QuickSort (A1rq):** Гибридный алгоритм. Выполняет MSD Radix Sort, но как только размер сортируемого подмассива становится меньше мощности алфавита (74), алгоритм переключается на Ternary String QuickSort. Это решает проблему накладных расходов MSD. Ожидается, что этот алгоритм покажет наилучшее время на практике.

2 Методология эксперимента

Для проведения замеров были реализованы классы `StringGenerator` и `StringSortTester`.

Генерация данных: Алфавит генератора состоит из 74 символов (A-Z, a-z, 0-9 и спецсимволы). Сгенерированы массивы размером $N \in [100; 3000]$ с шагом 100. Длина строк $L \in [10; 200]$. Типы массивов:

1. Случайные строки.
2. Почти отсортированные строки (получены из отсортированного массива путем небольшого числа перестановок).

3. Обратно отсортированные строки.

Измерения: Замеры проводились с использованием встроенных библиотек для работы со временем. Для минимизации погрешностей каждый алгоритм запускался несколько раз на одном и том же массиве, после чего бралось минимальное время. Количество посимвольных сравнений подсчитывалось с помощью счетчика, который инкрементировался при каждом обращении к символу.

3 Результаты и анализ

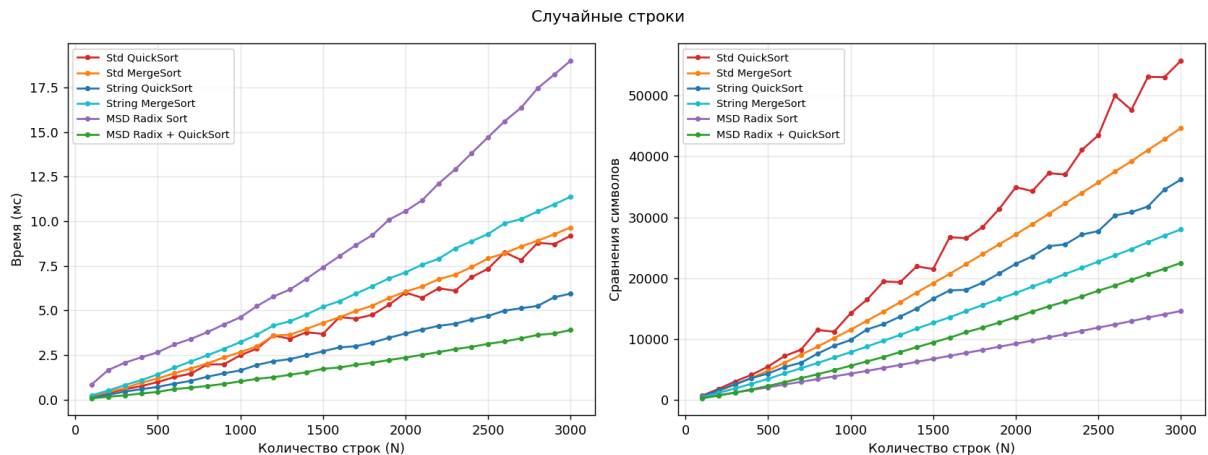


Рис. 1: Зависимость времени работы и количества посимвольных сравнений от размера массива для случайных строк

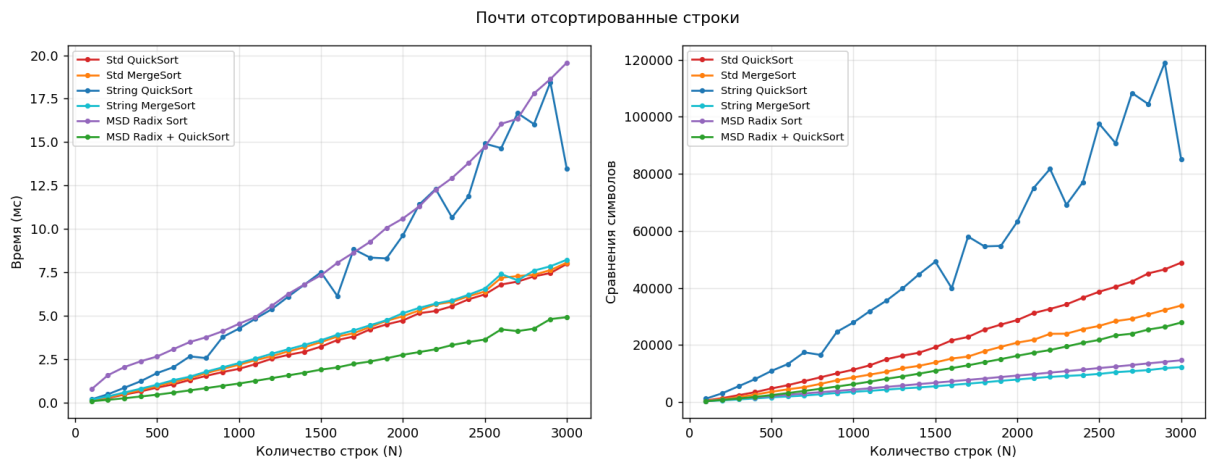


Рис. 2: Зависимость времени работы и количества посимвольных сравнений от размера массива для почти отсортированных строк

3.1 Сравнительный анализ

На основе собранных эмпирических данных можно сделать следующие выводы:

- **Случайные данные:** Гибридный **MSD Radix + QuickSort** показал наилучшее время работы. MSD Radix Sort делает меньше всего посимвольных сравнений, однако из-за накладных расходов на выделение корзин уступает гибриду по реальному

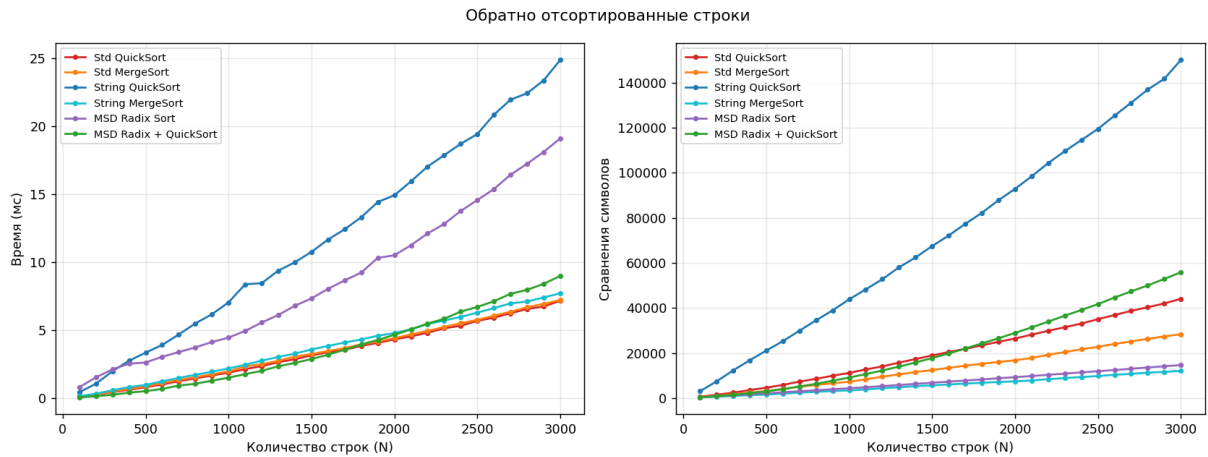


Рис. 3: Зависимость времени работы и количества посимвольных сравнений от размера массива для обратно отсортированных строк

времени. String MergeSort с LCP заметно сокращает число сравнений по сравнению со стандартным MergeSort.

- **Почти отсортированные данные:** Ternary String QuickSort с фиксированным опорным элементом `a[10]` деградирует на упорядоченных входах — разбиение становится несбалансированным, что приводит к взрывному росту числа сравнений (видно на рис. 2 справа). Наилучшее поведение по числу сравнений показывают **String MergeSort с LCP** (за счёт переиспользования длин общих префиксов) и **MSD Radix Sort**. По времени лидирует гибрид **MSD + QuickSort**.
- **Обратно отсортированные данные:** Ternary String QuickSort продолжает деградировать. MSD Radix Sort и String MergeSort с LCP остаются минимальными по числу сравнений (Std MergeSort также эффективен, поскольку на отсортированных данных слияние двух подмассивов выполняется почти без перекрытия). По времени стандартные QuickSort и MergeSort работают быстрее всего; гибрид **MSD + QuickSort** и String MergeSort показывают близкие к ним результаты.
- **Общий вывод:** Гибридный MSD Radix + QuickSort стабильно эффективен на случайных и почти отсортированных данных. String MergeSort с LCP даёт минимум посимвольных сравнений во всех вырожденных случаях. Ternary String QuickSort с базовым выбором опорного элемента подтверждает теоретическую уязвимость к упорядоченным входам.

4 Исходный код и проверяющая система

Ссылка на публичный репозиторий с исходным кодом (классы `StringGenerator`, `StringSortTester` и замеры):

https://github.com/Adaev1/Algorithms/tree/main/Set_9/A1

ID успешных посылок в системе CodeForces:

- A1m (String Merge Sort): *375940928*
- A1q (String Quick Sort): *375943421*
- A1r (MSD Radix Sort): *375958037*

- A1rq (MSD Radix + Quick Sort): *375964182*